Monocular Depth Estimation Implementation Progress Report

2026-10-19 | Phases 1-5 | Hardware-adapted implementation

Current result: a working RGB-only depth-perception prototype with real driving-video replay, relative depth maps, colour overlays, saved outputs, portable input verification and repeatable performance profiling.

The project addresses the absence of explicit depth information in ordinary camera images. A pretrained neural network estimates a dense relative-depth map from each RGB frame, providing a perception research prototype that runs on the available laptop hardware.

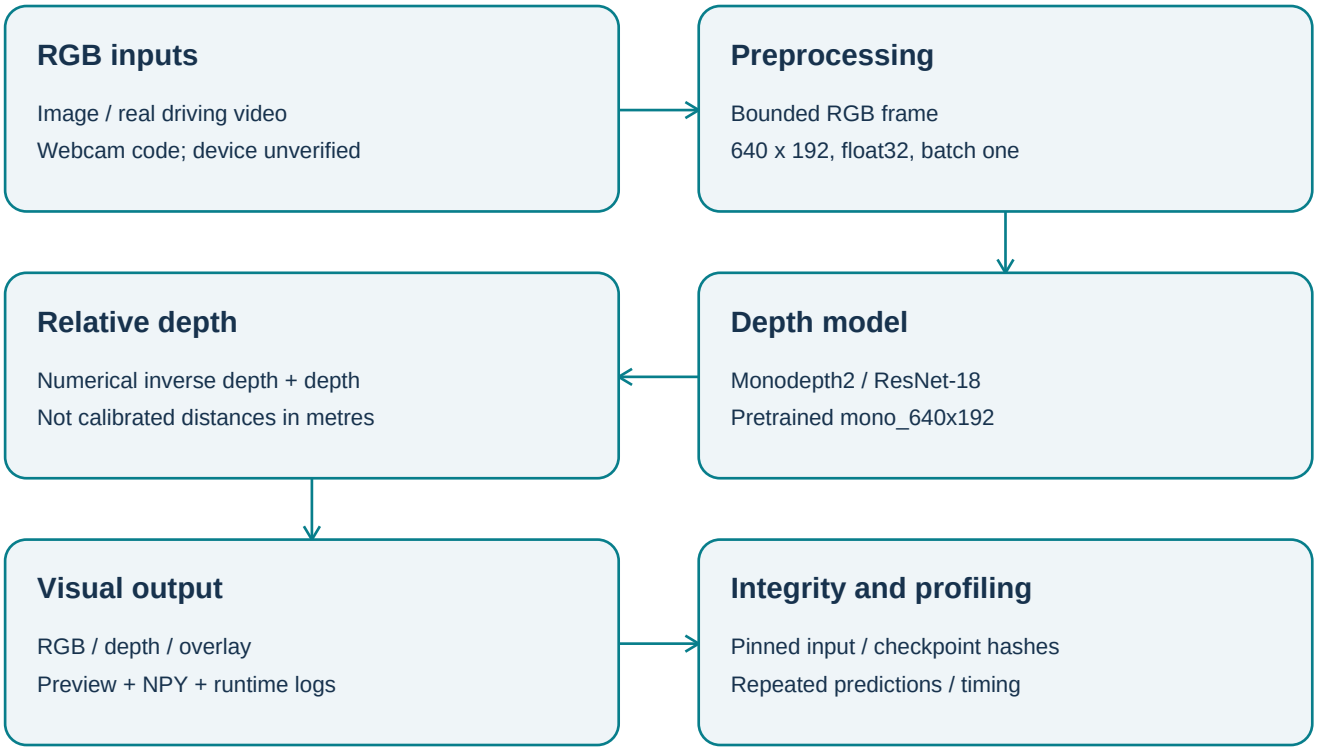
Phase	Implementation position
1 - Foundation	Image/video inference, RGB preprocessing, model loading and shared visualization implemented and tested.
2 - Demo inputs	Real driving sample, retained license, image fallback, portable manifest and checksum verification implemented.
3 - Stability / performance	Warmup-aware repeated profiling, numerical stability gate and measured rendering optimization implemented locally.
4 - Evaluation evidence	Paired RGB-D diagnostic, displayed runtime gates, latency statistics and visual-review exports implemented. Full driving benchmarks remain pending.
5 - Delivery	Integrated offline acceptance runner, rebuilt-environment validation, displayed rehearsal and presenter handoff implemented. Presentation-machine acceptance remains pending.

Scope: the latest hardware-adaptation instruction relaxes the original PDF implementation constraints. This report records implemented and tested behavior, not completion of every original PDF deliverable.

Model weights remain pretrained. No project-specific training, validated native CARLA camera run, calibrated metre-scale prediction or autonomous control is claimed.

Source baseline at generation: commit 5e0db26; Phase 5 delivery documentation is the current working-tree update. The delivery summary freezes Python source and requirements SHA256 hashes. Generated datasets and raw runtime evidence remain local.

1. Architecture and model



The webcam and optional CARLA interfaces feed the same preprocessing and inference path, but their real hardware integrations remain unverified. The verified inputs are a saved RGB image and real driving-video replay.

Component	Implemented contract
Encoder	ResNet-18; encoder.pth; strict loading of official checkpoint parameters.
Decoder	Monodepth2 multi-scale depth decoder; depth.pth; scale-zero output used for inference.
Checkpoint	mono_640x192; KITTI-trained monocular baseline; immutable recorded hashes.
Input	RGB HWC uint8 becomes a 1 x 3 x 192 x 640 float32 tensor in [0,1]. Original encoder normalization is retained.
Output	Positive finite inverse-depth and relative-depth maps, 192x640. Depth is the reciprocal of inverse depth.
Execution	Evaluation mode and torch.inference_mode; batch one; automatic CUDA/CPU selection.

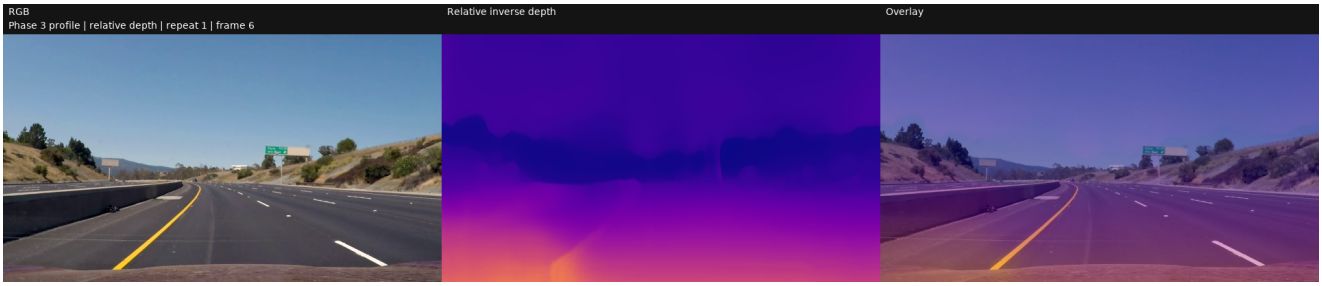
Module locations: src/model.py, src/preprocess.py, src/visualize.py, src/metrics.py and optional src/carla_capture.py. Phase 2 adds src/demo_data.py; Phase 3 adds src/profiling.py. project.py is the shared command-line entry point.

2. Working features and input preparation

Phase 1: reproducible dependency pins; official source/checkpoint acquisition and provenance checks; saved-image prediction; video/webcam input adapters; RGB/depth/overlay output; first-frame numerical arrays and preview; runtime logging; optional simulator capture/replay tools.

Phase 2: the prepare-demo command obtains a pinned Udacity driving sample and license. Every frame is decoded to check its count and dimensions, and a first-frame image fallback is saved. verify-demo validates the media, fallback image, baseline checkpoint and preprocessing identity after transfer to another laptop.

Frozen demo input	Recorded value
Source	Udacity / CarND-Advanced-Lane-Lines / project_video.mp4
Repository revision	91b080357290da37f533e2d0832f601c3e770d51
Video	1260 frames; 1280x720; 25 FPS; 50.4 source seconds
License	Repository MIT license retained as UDACITY_LICENSE.txt.
Ground truth	No aligned metric-depth ground truth is supplied by this sample.



Actual profile preview from the driving clip: RGB, relative inverse depth and overlay. Colours visualize model predictions; they do not establish measured accuracy.

Demo outputs: preview.png, inverse_depth.npy, relative_depth.npy, runtime.csv, summary.json and metrics.json. Without ground truth, metrics.json is empty and the summary explicitly marks accuracy unavailable.

Video replay is sequential. A processing cap below the native 25 FPS slows playback; it is not an interactive simulator or a guaranteed native-speed live stream.

3. Phase 3: stability and performance

Profile hardware: **NVIDIA GeForce GTX 1650**, Windows, Python 3.8.20, PyTorch 2.0.1+cu118. Input 640x192, batch one and two CPU threads. Each of 2 repeats excludes 5 full-pipeline warmup frames and measures the same 60 subsequent frames.

Numerical stability: every measured depth/inverse-depth pair is checked for shape, dtype, positivity, finiteness and reciprocal consistency. Both maps are hashed. All measured frame IDs and exact output fingerprints matched across the repeated runs on this machine.

Timing component	Median (ms)	P95 (ms)
Decode / RGB conversion / resize	5.75	8.07
Model only	22.40	28.50
Preprocessing + model + output transfer	30.37	38.63
Composite rendering	22.52	30.93
Numerical checks / hashing	2.18	2.74
Full measured headless cycle	61.30	78.42

Processing includes model time: these rows overlap and must not be added together. The measured cycle excludes asset verification, setup, warmup, display, pacing and output-file writes.

Repeat	Uncapped headless FPS	Measured frames
1	15.75	60
2	16.00	60

Rendering change: cache the palette, font and static labels; blend the overlay with installed OpenCV. A separate warmed 20-call rendering check measured median 55.06 ms before and 31.49 ms after (42.8% lower in this observation). Timing varies with system load; this is not a universal speedup guarantee.

The colour palette and numerical predictions are preserved. Overlay rounding may differ by one uint8 colour level. No model weights, preprocessing normalization or training configuration were changed.

4. Validation evidence and phase handoff

61 tests passed in the integrated delivery run. The regression suite covers Phases 1-5, including reference-data integrity, metric protocol, repeatability, displayed-duration gates, review exports, fail-fast delivery orchestration and source-change rejection.

Area	Evidence covered
Numerical / geometry	RGB order, tensor contract, depth decoding, intrinsics resizing, metric formulas and fixed colour scaling.
Safety / integration	Archive traversal/size checks, checkpoint hashes, frame synchronization, replay validation and cleanup paths.
Phase 2 transfer	Portable manifests, changed inputs/checkpoints, invalid metadata/paths, short decoding and source mismatch.
Phase 3 stability	Timing percentiles, warmup exclusion, finite reciprocal maps, repeat mismatch rejection and palette/overlay fidelity.
Phase 4 evaluation	Reference units/alignment/hash checks, masking/crop/resize order, equal-image aggregation, partial failures and duration-gate rejection.
Actual model	Official-checkpoint repeatability and real driving-video CUDA profiling, separate from mocked tests.

Earlier displayed rehearsal: 350 real-video frames over 71.58 seconds at 4.89 FPS under a 5 FPS cap. This covers 14 seconds of source footage. It is a different workload from the uncapped headless Phase 3 profile; the two FPS values are not directly comparable.

Final-profile process peak working set: 631.95 MiB. PyTorch peak allocated GPU memory: 106.34 MiB. Working set is not total system RAM or committed memory; PyTorch allocations are not total VRAM usage.

Transition gate: keep the accepted baseline and input hashes unchanged, run the full suite, verify-demo, image inference and a short video smoke test before accepting a later phase. Write into new output directories. A failed optional experiment must not replace the working baseline.

Evidence locations (local, ignored): runs/phase5_delivery/tests.log (current suite); runs/phase4_tests.json (historical suite); runs/phase3_profile_final/{summary.json,runtime.csv,fingerprints.json,preview.png}; runs/phase3_render_before.json and runs/phase3_render_final.json.

5. Phase 4: paired-depth diagnostic

Dataset: Open3D SampleRedwoodRGBDImages. Evaluated 5 correlated indoor reference frames from 1 scene, with 1,340,711 valid pixels in total. This is a small diagnostic set, not the KITTI Eigen or NYU benchmark.

The Open3D public sample archive is checksum-pinned. Its documented integer-millimetre depth images are converted to float32 metres. Reference depth is read only for scoring; model inference still receives RGB alone. Source, conversion, paired-asset hashes, split and checkpoint identity are frozen in the input manifest.

Protocol: registered RGB/depth dimensions; no crop for this reference; finite GT pixels strictly between 0.001 and 80 metres; OpenCV bilinear inverse-depth resizing before inversion; per-image GT median scaling; prediction clipping to [0.001,80]; equal-weight averaging of per-image metrics. Invalid or interrupted runs cannot report complete aggregate accuracy.

Metric	Diagnostic mean
Absolute relative error	0.4960
Squared relative error	1.1006
RMSE after GT median alignment (m)	1.4404
Natural-log RMSE	0.5071
Delta < 1.25	0.3763
Delta < 1.25 squared	0.6680
Delta < 1.25 cubed	0.8449

Finding: this indoor diagnostic has substantial prediction error. It demonstrates an evaluation workflow and exposes a limitation; it does not establish driving accuracy or satisfy KITTI targets. The cause has not been isolated. No fine-tuning or test-set selection was used to improve these scores.

Median alignment uses reference depth unavailable at deployment. An aligned RMSE in metres does not mean the RGB-only model predicts calibrated metre-scale distance. Do not attach these indoor scores to the separate driving-video demonstration.

Evidence: runs/phase4_reference_evaluation/{summary.json,per_sample.csv,*_relative_depth.npy,*_preview.png}; data/evaluation/redwood/evaluation_manifest.json. Full protocol and source links: docs/evaluation_protocol.md.

Diagnostic commands (PowerShell, fresh output folder):

```
.\.venv\Scripts\python.exe project.py prepare-reference-eval
.\.venv\Scripts\python.exe project.py evaluate --device cuda `
  --output runs/reference_evaluation_01
```

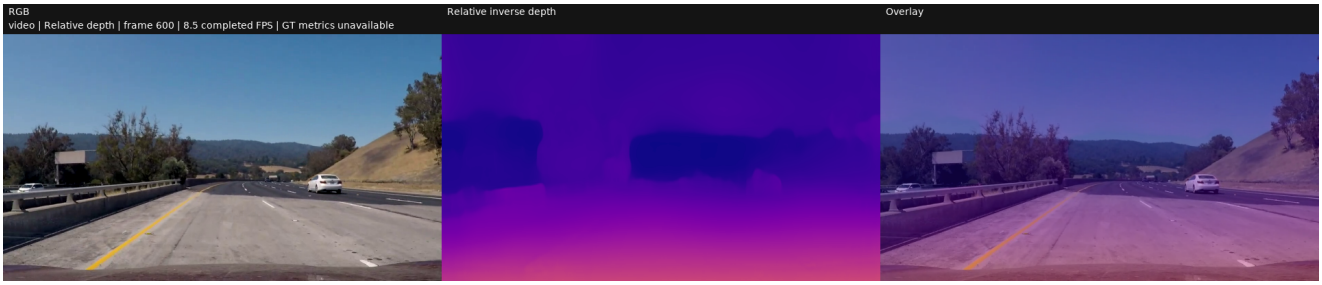
6. Phase 4: displayed runtime and review

Both runs used the frozen real driving clip and the local GTX 1650, with a visible window and a 60-second minimum-duration gate. No model weights or preprocessing were changed. Review snapshots and first-frame overhead are included in frame-loop timings.

Displayed run	Frames / seconds	Observed FPS	Configured cap
Capped rehearsal	650 / 76.26	8.52	10
Full source clip	1260 / 71.40	17.65	30

The source is 25 FPS. A configured cap of 30 is effectively limited by the source to 25 FPS. Lower processing throughput slows sequential replay. These observations are not webcam/CARLA performance or cross-laptop guarantees.

Full-clip stage	Median (ms)	P95 (ms)
Model	19.56	24.89
Preprocessing + model + output	25.26	33.20
Arrival to display	46.47	66.13
Frame cycle, including pacing	50.34	72.95



Inspected adjacent pairs 200/201 and 600/601: broad road-to-horizon structure is visible; vehicle boundaries are coarse. Motion changes both RGB and predicted maps. These snapshots are qualitative evidence, not a motion-compensated flicker metric or ground-truth accuracy test.

Full-clip process peak working set: 734.73 MiB; PyTorch peak allocated GPU memory: 106.34 MiB. Total system/driver memory peaks were not recorded. The friend's RTX 3050 remains a separate acceptance gate.

Saved review.json maps frame IDs to RGB, inverse-depth NPY and preview files. A short displayed run fails the duration gate; headless runs cannot satisfy it. Existing commands still work without the optional review/gate flags.

7. Phase 5: integrated delivery acceptance

tools/validate_delivery.py runs the existing phase commands sequentially with the same Python interpreter and frozen default baseline. It performs no downloads, creates exclusive output directories, saves per-step logs and stops on failure. Source hashes must remain unchanged throughout the run.

Check	Recorded result
tests	pass (169.44 s)
inputs	pass (12.28 s)
environment	pass (14.71 s)
image	pass (13.10 s)
profile	pass (28.78 s)
evaluation	pass (24.35 s)
video	pass (21.99 s)
display	pass (117.55 s)

Final displayed rehearsal: 650 frames in 82.55 seconds, 7.87 FPS including the display loop under a 10 FPS cap. The minimum 60-second displayed gate passed. This is real driving-video replay with newly computed predictions, not a native CARLA simulation.

The short repeated profile passed with exact frame predictions matching: True. Image inference, video smoke and the five-frame reference evaluator passed through their original interfaces. The baseline weights and preprocessing were retained.

Environment reproduction: a separate project-local Python 3.8.20 environment was rebuilt from all 30 pinned requirements using the local package cache. The integrated run used that rebuilt interpreter. This verifies local reproduction, not installation or performance on the RTX 3050 laptop.

Presentation handoff: docs/demo_guide.md contains fresh-machine setup, offline asset transfer with licenses, the accepted video command, webcam/image fallbacks, a short explanation script and remaining hardware gates. Keep raw run summaries with the handoff separately from Git.

Acceptance status: software delivery track validated locally. Academic submission, target-laptop acceptance, physical camera, native simulator, full benchmark accuracy and any project-specific training remain separate outstanding work.

8. Remaining work and reproducible usage

Requirement	Honest current position
Fine-tuning	Not performed. No prepared calibrated temporal training dataset or local pose/loss training pipeline; pretrained baseline retained.
Ground-truth accuracy	Small reference diagnostic completed; full KITTI/CARLA accuracy targets remain unvalidated.
Live simulator / webcam	Native CARLA sensor validation and a physical-camera run remain pending.
Presentation laptop	RTX 3050 testing remains pending; current measurements belong to the local GTX 1650.
Runtime and metric scale	Displayed video throughput is recorded above; native-speed camera/CARLA behavior and calibrated metre-scale depth remain unvalidated.
Vehicle control	Path planning, steering and throttle control are outside the original PDF scope and are not implemented.

Run from the project directory in PowerShell (choose unused output names):

```
$depthPython = '.\.venv\Scripts\python.exe'
& $depthPython project.py assets
& $depthPython project.py prepare-demo
& $depthPython project.py verify-demo
& $depthPython -m unittest discover -s tests -v
& $depthPython project.py profile --device cuda --frames 60 `
  --warmup 5 --repeats 2 --output runs/profile_review_01
& $depthPython project.py demo --source video `
  --video data/demo/driving.mp4 --manifest data/demo/input_manifest.json `
  --device cuda --frames 350 --fps 5 --output runs/demo_review_01
```

On the friend's laptop, transfer matching source, checkpoints, pinned third-party source and data/demo, then rebuild .venv. Verify CUDA, hashes and tests; rehearse the displayed demo there. Do not copy .venv or the CARLA runtime from the development machine.

References and provenance

Official Monodepth2 source and pretrained models

Udacity driving sample and repository license

Open3D RGB-D reference format and units

Project repository

Original scope reference: monocular_depth_report_FINAL_FIXED (1).pdf, Sections 3.2-3.3 and Chapter 6. Current phase plans and repository code distinguish completed work from pending original targets.